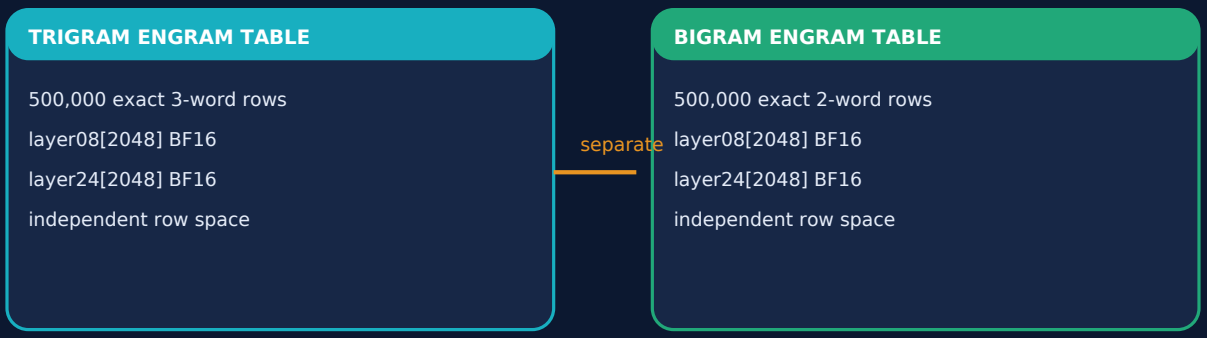
DENDRITRON RECURRENT ENGRAM

Sparse Memory Carries Knowledge. Two Blocks Carry Thought.

A CPU/ARM-first architecture built from exact phrase memory, layer-2 concept geometry, LoRA skills, experts, and branches



Primary intent: pay for the donor once.
Retrieve knowledge sparsely during live CPU/ARM execution.

Technical architecture brief | Revision 3.0 | 3 August 2026

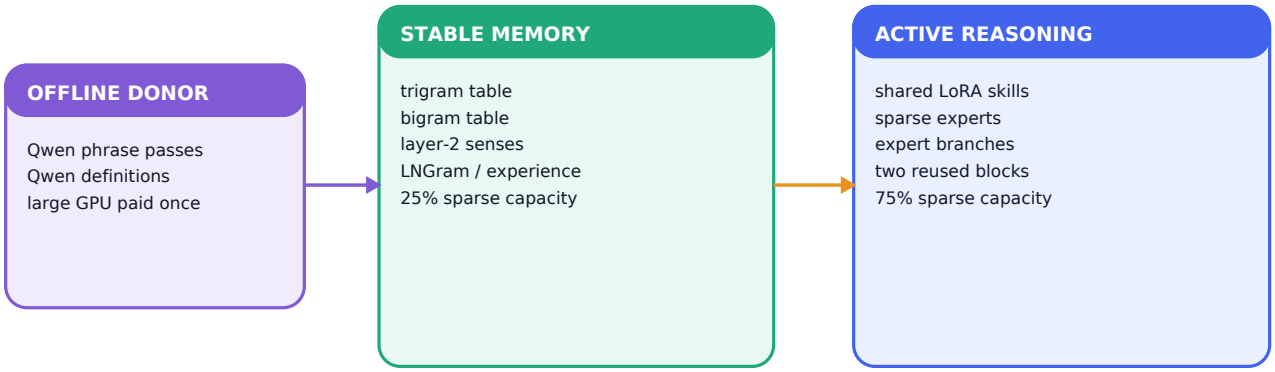
Architecture synthesis and implementation: Ryan Carson

01 | INTENT

Move knowledge out of repeated computation

ARCHITECTURE THESIS

Dendritron uses a large donor to construct semantic memory once, then gives a smaller CPU/ARM recipient sparse access to those learned states while two recurrent blocks perform the changing reasoning.



LIVE TARGET: CPU / ARM

Stable knowledge and active thought have different jobs. Exact phrase vectors and definition-sense points form addressable reference memory. Shared LoRA skills, experts, typed branches, and repeated block visits transform the live hidden state.

- The donor cost occurs during offline phrase and definition extraction.
- The live path fetches selected rows and activates a bounded compute neighborhood.
- The fixed sparse-capacity ledger assigns 25% to memory and 75% to conditional compute.

RESEARCH BASIS AND DENDRITRON EXTENSION

DeepSeek's Conditional Memory via Scalable Lookup separates static retrieval from dynamic computation. Memory Grafting turns offline donor hidden states into frozen exact n-gram values. Carson combines these with concept geometry, LoRA skills, experts, branches, and a two-block CPU/ARM recipient.

02 | OFFLINE MEMORY

The two Engram tables and what every row stores

TWO PHYSICAL TABLES - EACH HIT RETURNS TWO DONOR DEPTHS

TRIGRAM ENGRAM

key: exact 3-word phrase
rows: 500,000
row i has its own shard pointer
payload A: layer08[2048]
payload B: layer24[2048]
text + frequency retained

BIGRAM ENGRAM

key: exact 2-word phrase
rows: 500,000
row j has its own shard pointer
payload A: layer08[2048]
payload B: layer24[2048]
text + frequency retained

one row = address metadata + two 2,048D BF16 vectors

Table	Address	Rows	Frozen payload
Trigram Engram	Exact three-word phrase	500,000	UTF-8 phrase, frequency, layer08[2048] BF16, layer24[2048] BF16
Bigram Engram	Exact two-word phrase	500,000	UTF-8 phrase, frequency, layer08[2048] BF16, layer24[2048] BF16

OFFLINE EXTRACTION CONTRACT

```
isolated phrase -> one Qwen forward pass
final non-padding subtoken -> hidden_states[8]
final non-padding subtoken -> hidden_states[24]
write both 2,048D vectors into that phrase row
```

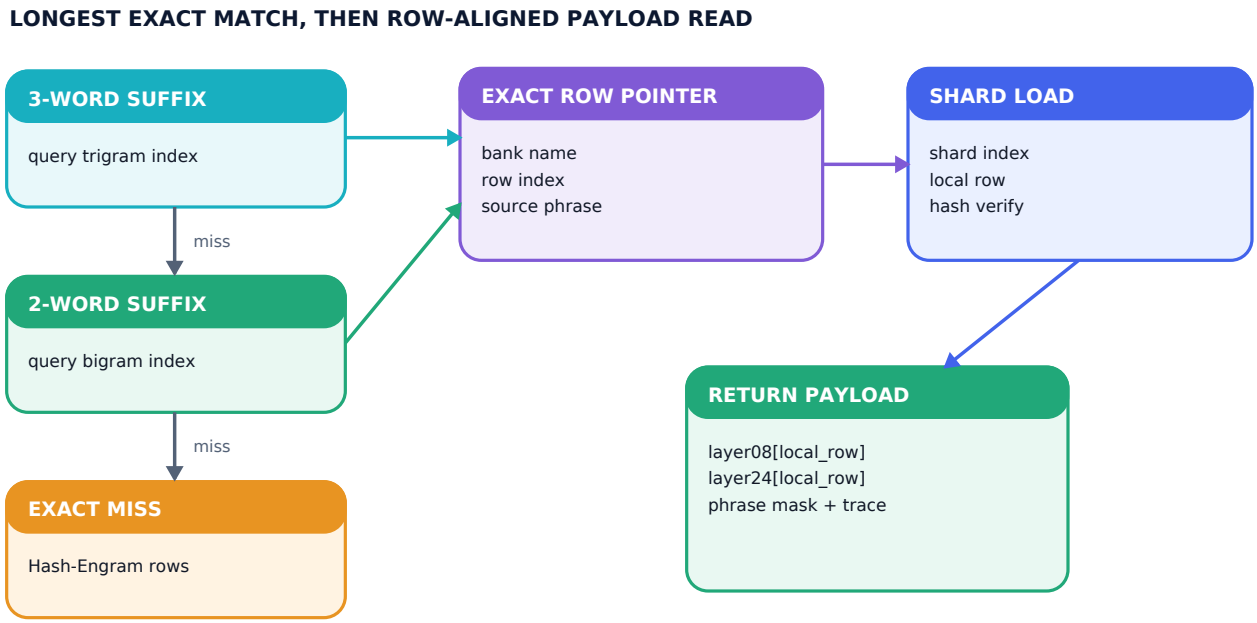
Why these depths: layer 8 preserves earlier lexical and compositional construction; layer 24 preserves deeper composed phrase meaning. Their paired views let the recipient use knowledge at two stages of its own reasoning rather than inherit a single late donor summary.

RESEARCH BASIS AND DENDRITRON EXTENSION

Memory Grafting supplies final-token donor-state storage. Dendritron extends one donor view into two stored depths per row and keeps bigram and trigram rows in independent physical tables.

03 | LIVE PHRASE LOOKUP

From exact suffix to both hidden-state vectors



Hidden states are read from memory; the donor model stays offline.

LONGEST-MATCH ADDRESS AND PAYLOAD READ

```
if exact three-word suffix exists:
    pointer = (trigrams, row_i)
elif exact two-word suffix exists:
    pointer = (bigrams, row_j)
else:
    activate trainable Hash-Engram miss rows
layer08, layer24 = frozen_store.get(pointer)
```

The pointer selects a bank, shard, and local row. The loader verifies the immutable shard and returns **both** payload tensors. Phrase text and IDs stay attached as address and trace metadata; vector coordinates carry the donor's learned representation.

Layer 8 enters initialization and early recurrent visits through its JTD map. Layer 24 enters physical Block 2 through its own JTD map, giving the contraction stage a deeper phrase view.

RESEARCH BASIS AND DENDRITRON EXTENSION

DeepSeek Engram supplies deterministic lookup and hash miss coverage. Memory Grafting supplies longest exact suffix priority. Dendritron's row-aligned store reads paired layer-8 and layer-24 values from the selected bigram or trigram row.

04 | CONCEPT RESOLUTION

LNGram -> JTD -> layer-2 kNN -> one active sense

Exact phrase lookup answers **which phrase occurred**. The concept route answers **which meaning is active**. The evolving hidden state becomes a hard latent 2/3-gram LNGram address. That address restricts the concept region. Separate JTD maps then place the live state and both phrase views into the frozen Qwen layer-2 definition geometry.

Euclidean k-nearest-neighbor retrieval selects one active definition-sense point in the routed region. The definition's word ID, sense ID, exact source text, and ordered definition-word links remain pointers and trace evidence. The layer-2 vector remains the concept location.

IMPLEMENTATION STATUS

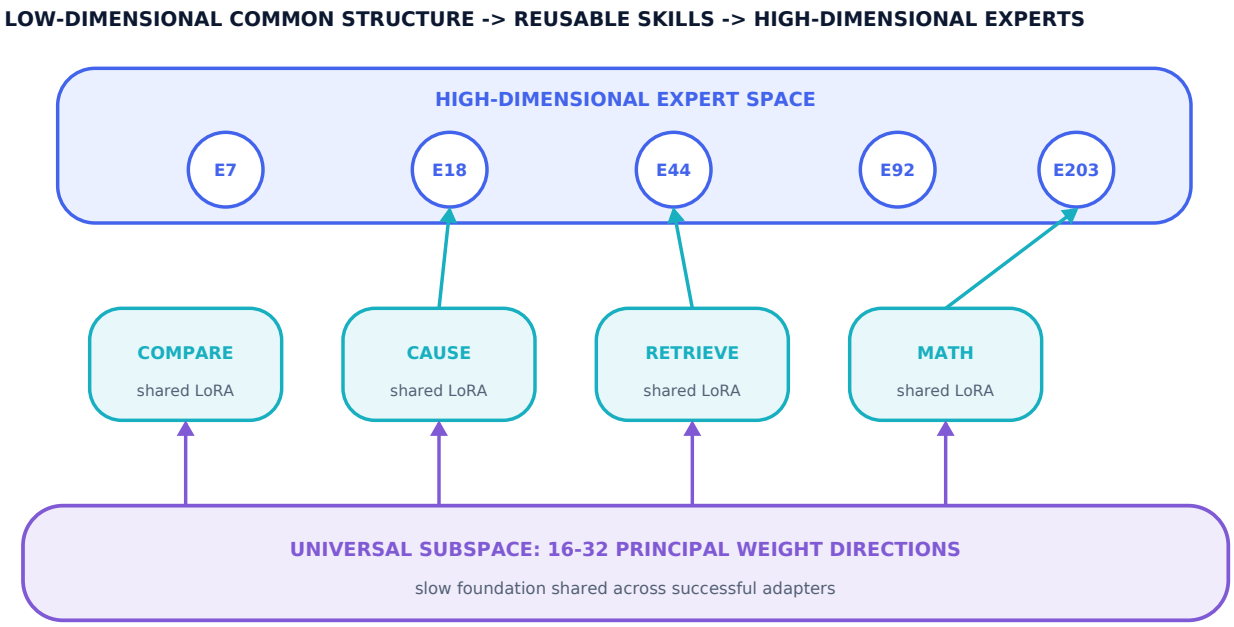
Current code boundary: the reference implementation already maps layer 8, layer 24, and live states into the layer-2 frame and weights exact word-sense candidates by Euclidean distance. The final LNGram-routed concept-region kNN executor that returns one sense remains to be connected.

RESEARCH BASIS AND DENDRITRON EXTENSION

Lngram supplies hidden-state discretization and exact latent n-gram lookup. Locality Preserving Joint Transfer supplies source-specific maps and neighborhood preservation. Carson fixes layer-2 definition senses as landmarks and connects the latent address to one-sense Euclidean retrieval.

05 | OPERATION HIERARCHY

Universal directions, shared LoRA skills, and experts



Level	What it is	Why it exists
Universal subspace	About 16-32 principal directions in low-rank weight-update space	A slow common coordinate system shared across successful adapters
Shared LoRA skills	Learnable low-rank adapters extending the common directions	Reusable operations shared across tasks and experts
Experts	High-dimensional conditional modules outside the low-rank basis	Specialized task/domain junctions and branch ownership

The router compares the current task with a small skill bank and selects the smallest sufficient skill set. Skill adjacency opens a bounded expert neighborhood. The expert library can grow while the global geometric comparison remains skill-sized.

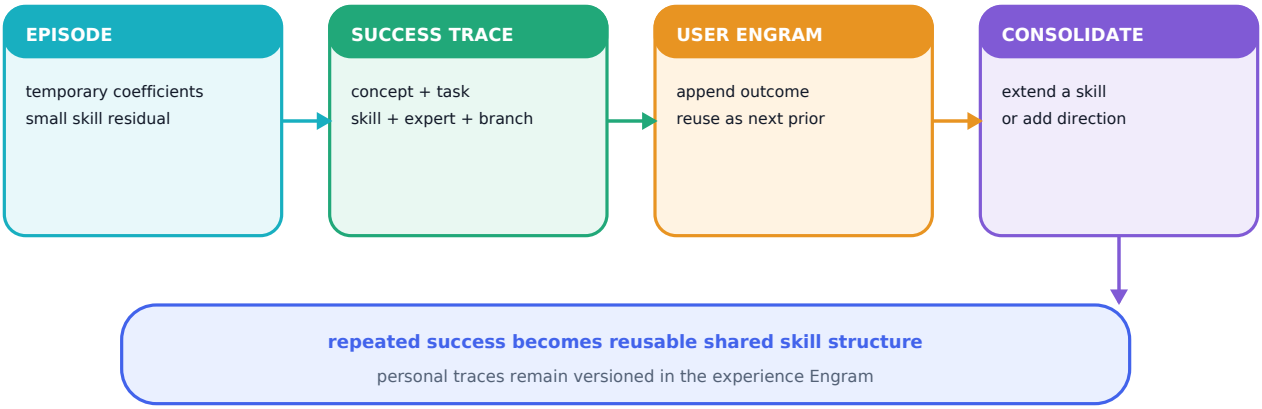
RESEARCH BASIS AND DENDRITRON EXTENSION

Shared LoRA Subspaces for Almost Strict Continual Learning supports an evolving foundational adapter subspace and compact task coefficients. Dendritron turns reusable adapters into skills and places high-dimensional experts beyond that low-rank foundation.

06 | CONTINUAL LEARNING

How successful user adaptation becomes LoRA Engram memory

USER LEARNING STAYS LOCAL FIRST, THEN CONSOLIDATES WHEN IT REPEATS



A user interaction first trains temporary coefficients over a few active shared skill directions. A successful outcome records its concept IDs, task relation, skills, experts, branch trace, and evidence. That record enters a versioned user experience Engram and becomes a prior when the same neighborhood returns.

Repeated, generalizable residual structure is consolidated into an existing shared skill. Persistent orthogonal structure can add a new skill direction. The experience tier preserves personal adaptation; the shared skill tier preserves reusable operations; the universal foundation changes at the slowest cadence.

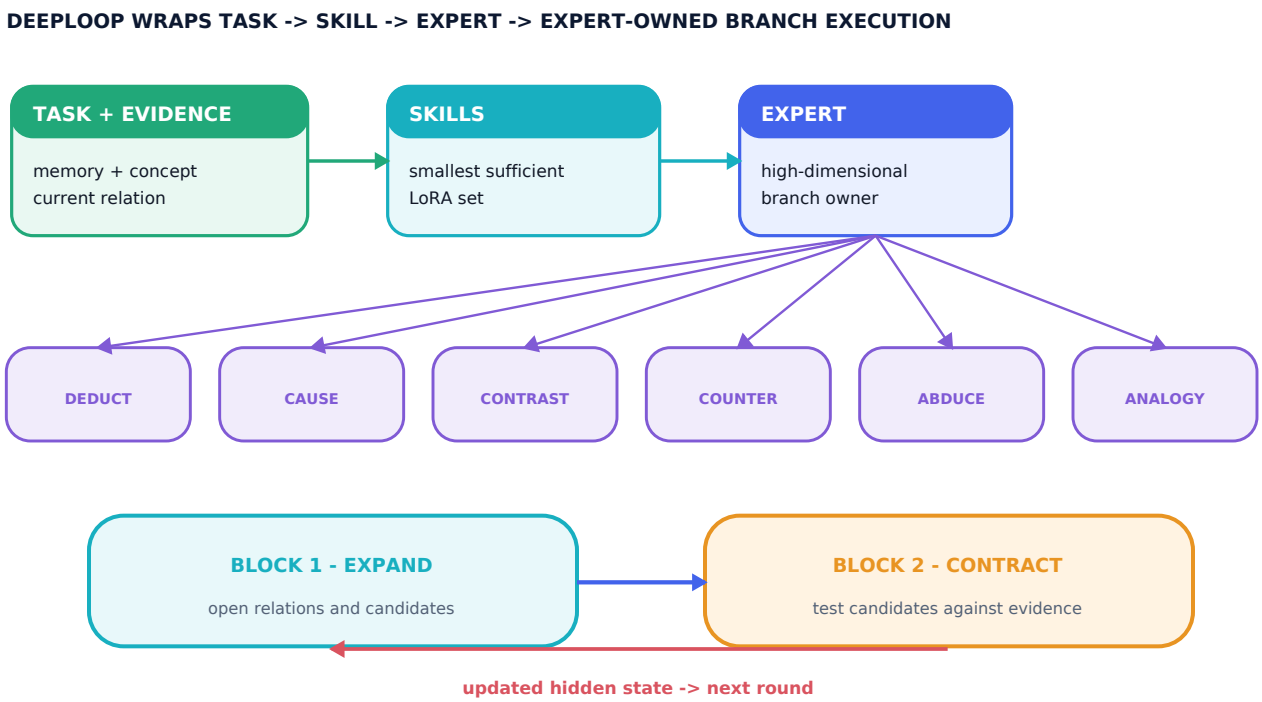
State	Update speed	Stored knowledge
Episode coefficients	Fast	Current user/task adjustment
Experience Engram	Append after verified success	Reusable personal outcome and route trace
Shared skill LoRA	Consolidation	Operation that generalizes across episodes
Universal directions	Slow structural revision	Common low-rank foundation

IMPLEMENTATION STATUS

Experience commits and consolidation are target-architecture modules. The repository contains their schemas and shared-subspace utilities; the complete per-user runtime remains to be connected.

07 | REASONING ENGINE

Task -> skill -> expert -> branch inside the two-block DeepLoop



Expert-owned branch	Bound operation
Deductive	Premises -> candidate conclusion -> support and contradiction test
Causal	Cause -> mechanism -> effect under causal order
Contrastive	Preserve dimensions that separate competing candidates
Counterfactual	Change one condition and propagate the differences
Abductive	Rank explanations against observations and contradictions
Analogical	Transfer relation structure between concept neighborhoods

Block 1 expands relations, candidate explanations, and possible conclusions. Block 2 contracts those candidates against phrase memory, the active layer-2 concept, causal visibility, and opposing evidence. The updated hidden state returns to Block 1 for the next round. Two stored blocks therefore provide effective depth **2R** across **R** rounds.

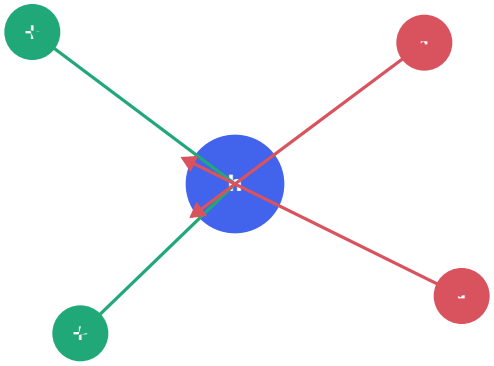
RESEARCH BASIS AND DENDRITRON EXTENSION

DeepLoop supplies loop-aware Post-RMSNorm scaling for repeatedly visited blocks. Carson assigns complementary expansion and contraction roles and places expert-owned semantic branches inside every recurrent round.

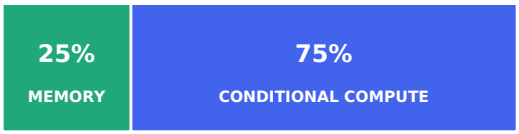
08 | GEOMETRIC CONTROL

HarMax evidence, 25/75 capacity, and CPU/ARM execution

SIGNED EUCLIDEAN EVIDENCE AND THE FIXED 25/75 CAPACITY LAW



c = evidence mass - distance mass



stable reference : changing region = 1 : 3

amplitudes 0.5 and sqrt(3)/2 square to 0.25 and 0.75

Each branch builds a small causal anchor pool. **p** is the mass implied by Euclidean distance; **y** is the mass assigned by bound evidence; **c = y - p** creates signed motion. Positive coefficients attract the thought toward underrepresented support. Negative coefficients repel it from contradictory or excess mass. The harmonic residual records unresolved evidence.

The 25/75 rule is an architecture constraint over scalable sparse capacity. Memory is the stable one-quarter reference; conditional compute is the three-quarter changing region. The amplitudes **0.5** and **sqrt(3)/2** independently square to **0.25** and **0.75**. Engram allocation results act as corroborating evidence.

CPU/ARM choice	Live effect
Offline donor extraction	Large-model semantic construction is paid once
Exact row lookup	Only selected bigram/trigram payloads move
Two reused blocks	Reasoning depth grows through visits rather than stored layers
Low-rank skills + sparse experts	A bounded conditional path executes per task
Euclidean geometry	HarMax, concept kNN, LNGram readout, and vocabulary ranking share one distance primitive

EXECUTION CONTRACT

Runtime bilinear operator count: 0. The production target uses memory-mapped BF16/INT8 rows, quantized linear kernels, vectorized Euclidean distance, and ARM-aware sparse fetch.

09 | HONEST BOUNDARY

What is built, what the architecture still requires

Subsystem	Current status
500k trigram + 500k bigram inventories	Reported complete; external tables are omitted from the public repository
Paired layer-8/layer-24 phrase payloads	Reported complete; immutable external assets
Exact 3 -> 2 routing and paired row loading	Implemented and tested with synthetic indexes/payloads
Layer-2 definitions, JTD maps, LNGram	Pipelines and runtime modules implemented; real data fit/integration remains
Two-block recurrent CPU reference	Forward/backward path and checkpoint reload recorded
Universal/skill/expert structural route	Generic trainable path implemented
One-sense concept-region kNN	Target executor specified; live connection remains
Typed semantic branches	Six contracts specified; live executors remain
User LoRA Engram consolidation	Schema specified; full runtime remains
Full language training and optimized ARM runtime	Experimental and engineering milestones

Immediate build order

- Connect LNGram-routed layer-2 kNN and return one active definition sense.
- Materialize the real phrase/definition payloads and fit the JTD maps.
- Implement one complete deductive expert branch with source trace.
- Extend the branch interface, then connect user LoRA commits and consolidation.
- Train and measure language quality, memory use, loop convergence, CPU throughput, and peak memory.

Primary references

[1] Cheng et al. Conditional Memory via Scalable Lookup. arXiv:2601.07372. [2] Cheng et al. Memory Grafting. arXiv:2605.20948. [3] Zheng et al. Lngram. arXiv:2605.24869. [4] Li et al. Locality Preserving Joint Transfer. arXiv:1906.07441. [5] Kaushik et al. Shared LoRA Subspaces. arXiv:2602.06043. [6] Li et al. DeepLoop. arXiv:2607.13491. [7] Lin. Collision-Free Hot-Tier Extension. arXiv:2601.16531.

CONTRIBUTION BOUNDARY

The original Dendritron notes supplied the architectural starting point and the CPU/ARM objective. This brief documents Ryan Carson's synthesis, extensions, reference implementation, and remaining experimental boundary.